PHP - Classes

Definição de classe

- Classes são conjuntos de variáveis e funções que descrevem um objeto com suas características e ações. Um objeto pode usar todas variáveis e funções existentes em sua classe

- **Classe apartamento (propriedades)**
  - numero_quartos
  - numero_banheiros
  - preco_aluguel
  - preco_venda
  - tamanho

Como criar uma classe

```
class nome_classe
{
        <variáveis>
        <procedimentos>
}
```

- *var* é usado para criar variáveis dentro de uma classe. Também pode ser usado a variável interna do PHP, *$this* que serve para referenciar o próprio objeto

- Para criar um novo objeto, deve ser atribuído a uma variável que representará o objeto a instrução *new* seguida do nome da classe

Como criar um classe

```
class Loja
{
        var $itens;
        function adiciona ($codigo,$quantidade){
                if(isset($this->itens[$codigo]))
                        $this->itens[$codigo] += $quantidade;
                else
                        $this->itens[$codigo] = $quantidade;
        }
        function remove($codigo,$quantidade){
                if(itens[$codigo] > $quantidade;){
                        itens[$codigo] -= $quantidade;
                }
                else{
                        return false;
                }
```

Como criar um classe

- Classe Loja
  - Possui como variável o array *itens* que foi declarada usando a instrução var
  - Também possui duas funções para acrescentar ou remover itens do estoque
  - O objeto é criado quando fazemos a atribuição de *new* Loja à $*estoque*

Como criar um classe

- Note que o uso do símbolo -> para acessar funções e variáveis. Sempre deve ser usado quando trabalhando com objetos

```
...

$estoque->adiciona("bermuda",2);
$estoque->adiciona("camiseta",3);

echo "A loja já possui " . $estoque->itens["bermuda"] . "bermudas. <br>";
echo "A loja já possui " . $estoque->itens["camisetas"] . "camisetas. <br>";

...
```

Palavra-chave *private* e *protected*

- Utilizadas para criar métodos e variáveis privadas e protegidas dentro de uma classe (PHP 5)
- Uma variável ou método privados só podem ser acessados de dentro da classe que foram declarados
- Uma variável ou método protegidos poderão

Palavra-chave *private* e *protected*

```
class Classe1{
        private $var1 = "Olá, var1! \n";
        protected $var2 = "Olá, var2! \n";
        protected $var3 = "Olá, var3! \n";

        function bomDia(){
                print "Classe1 ". $this->var1. "<br>";
                print "Classe1 ". $this->var2. "<br>";
                print "Classe1 ". $this->var3. "<br><br>";
        }
}

    class Classe2 extends Classe1{
            function bomDia(){
                Classe1::bomDia(); //Exibe
                print "Classe2 ". $this->var1. "<br>"; //Não exibe nada (erro)
                print "Classe2 ". $this->var2. "<br>"; //Exibe  print
                "Classe2 ". $this->var3. "<br><br>"; //Exibe

        }
```

Palavra-chave *private* e *protected*

```
class Classe1{
        private function MetodoPrivado(){
                echo "Classe1::MetodoPrivado() chamado.
                <br>";
        }
        protected function MetodoProtegido(){
                echo "Classe1::MetodoProtegido() chamado.
                <br>";
                $this->MetodoPrivado();
        }
}

class Classe2 extends Classe1{
        public function MetodoPublico(){
                echo "Classe2::MetodoPublico()
                chamado.<br>";
```

Métodos abstratos e interface

- O PHP 5 permite a criação de métodos abstratos, ou seja, o método é apenas declarado, sem uma implementação. Isso é feito por alguma classe que a herda

- A classe com o método abstrato também deve ser declarada abstrata e não pode criar objetos diretamente

Métodos abstratos e interface

```
abstract class ClasseAbstrata{
        abstract public function teste();
}

class ClasseImplementacao extends ClasseAbstrata{
    public function teste(){
        echo "Método teste() chamado!<br>";
    }
}

//Erro $obj = new ClasseAbstrata;
```

Métodos abstratos e interface

- O PHP 5 também possibilita a criação de interfaces. Recurso parecido com classes abstratas porém uma classe pode implementar diversas interfaces

- São adicionadas apenas as declarações dos métodos que irão fazer parte da classe que irá implementá-la

- A palavra reservada *implements* indica uma

Métodos abstratos e interface

```
interface MinhaInterface{
    public function Teste();
}

Class MinhaClasse implements MinhaInterface{
    public function Teste(){
        echo "Minha implementação de uma interface funciona!";
    }
}
```

Palavra-chave *final*

- Os métodos declarados com a palavra *final* não poderão ser sobrescritos pelas [illegible] erro causo ocorra

```
Class MinhaClasse implements
MinhaInterface{
        final function Teste(){
                        //..
        }
}
```

- Uma classe também pode ser *final*, não permitindo subclasse, causando erro na tentativa

```
final class
MinhaClasse {
      ///..
}
```

Construtores e destrutores

- O método *construct* é executado toda a vez que for criado um novo objeto da classe aonde foi declarado. Isso permite que objetos internos sejam iniciados antes do seu uso

- Por questões de compatibilidade, se o *construct* não for achado, o PHP irá procurar por um método que possua o

Construtores e destrutores

```
class Classe{
        function construct(){
                print "Este é construtor da Classe<br>";
        }
}

class SubClasse extends Classe{
        function construct(){
                parent:: construct();
                print "Este é construtor da SubClasse<br>";
        }
}
```

Construtores e destrutores

- Uma classe também pode ter um destrutor, usando destruct(). Será chamado toda a vez depois da última referência a objeto, antes da liberação de sua memória

- Útil para fins de depuração, fechamento de conexão com banco de dado entre outras tarefas

Construtores e destrutores

```
class Classe{
        function  construct(){
                $this->nome= "Classe com construtor e destrutor";
                print "Este é construtor da ". $this->nome ."<br>";
        }
        function  destruct(){
                print "Este é destrutor da ". $this->nome ."<br>";
        }
}
```

Variáveis e métodos estáticos

- Variáveis e métodos estáticos (antecedidos por *static*) podem ser usados mesmo sem que a classe seja iniciada

```
class Classe{
        static $variavel_estatica = 10;

        public static function Estatico(){
                echo "<br>Isso é um método estático!";
        }
}
```

Referências

- PHP Manual:
  - http://www.php.net/manual/pt_BR/index.php
- Desenvolvendo Websites com PHP
  - De Juliano Niederauer